Faculty of Computer Science

DATA SCIENCE AND BUSINESS ANALYTICS

Machine Learning 1

Support Vector Machines

Alexey Boldyrev

10/03/2026

Image credit: 'The Mayflower at Sea'
by Granville Perkins, 1876
Wallach Division Picture Collection
The New York Public Library.

Outline

Overview of Support Vector Machines

The SVM Family — increasingly sophisticated classifiers:

- **Separating hyperplane** — any boundary between two classes
- **Maximal Margin Classifier (MMC)**
  - Finds the **widest "street"** between classes
  - Requires **perfect linear separability**
- **Support Vector Classifier (SVC)**
  - Adds **soft margins** — tolerates some violations
  - Works with **non-separable** data
- **Support Vector Machine (SVM)**
  - Uses **kernel trick** for non-linear boundaries
  - Can model **complex** decision surfaces

Vladimir Vapnik
Photo: Robert Williamson

History: Developed by Vladimir Vapnik et al. from MMC (1963) → SVM with kernels (1993). Binary classifier; multi-class via OvO/OvA.

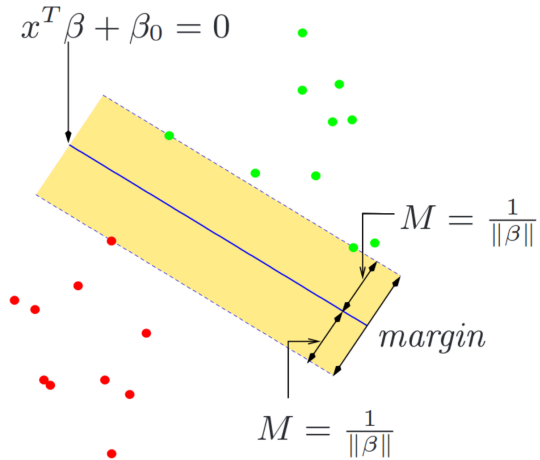
The Key Idea: Maximum Margin

The "Widest Street" Analogy (Andrew Ng):

- Don't just draw a line — draw a **street** between classes
- **Wider street** → more **confident** predictions
- **Widest possible street** = **maximal margin classifier**
- Points on street edges = **support vectors**

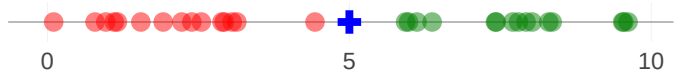
Why does wider margin help?

- Restricts set of possible hyperplanes
- Fewer hypotheses → lower VC dimension → **better generalization**
- **Margin maximization ≈ regularization**



Source: ESL Fig. 12.1

Hyperplane in 1D



Separating hyperplane does not always exist

- It's a point in 1D space for $\beta_i \in \mathbb{R}$,

$$P := \{x \mid \beta_0 + \beta_1 x = 0\} = \left\{x \mid \begin{bmatrix}\beta_0 \\ \beta_1\end{bmatrix} \perp \begin{bmatrix}1 \\ x\end{bmatrix}\right\} = \{-\beta_0/\beta_1\}$$

- A single point satisfies the set's condition

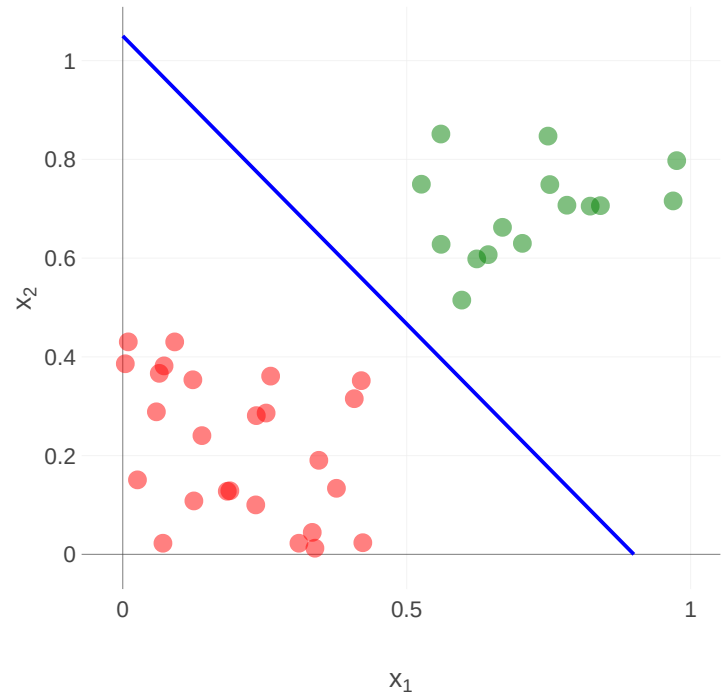
Hyperplane in 2D

Definition: A line in 2D space

$$L := \{x \mid \beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0\} \subset \mathbb{R}^2$$

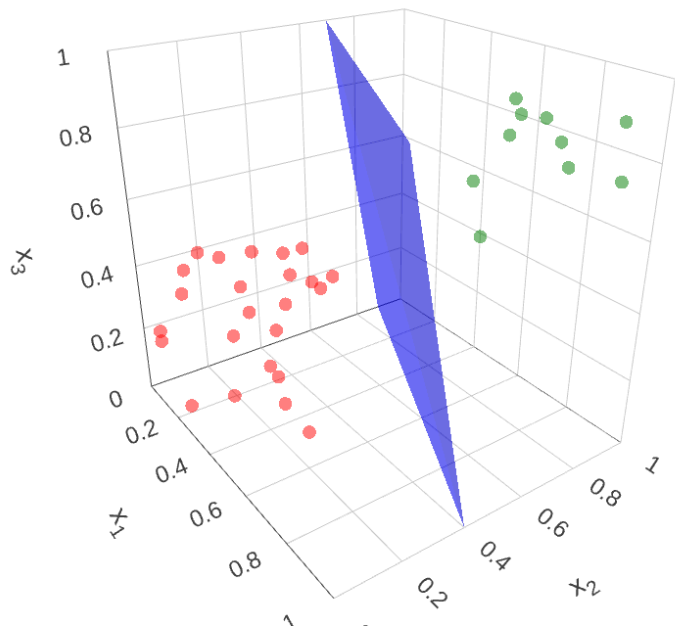
- Equivalent to: $x_2 = -\frac{\beta_0}{\beta_2} - \frac{\beta_1}{\beta_2} x_1$
- Vector $\vec{\beta} = (\beta_1, \beta_2)^T$ is **orthogonal** to the line
- β_0 shifts the line away from origin

Key insight: Points on opposite sides have opposite signs of $\beta_0 + \beta^T x$



Hyperplane in 3D

- It's a plane in 3D space, $H := \{x \mid \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 = 0\} \subset \mathbb{R}^3$



Hyperplane in p Dimensions

- We can't visualize the hyperplane for $p > 3$ 😵
- It's a $p-1$ dimensional object in p dimensional space, $\{x_{p\times 1} \mid \beta_0 + \beta_{p\times 1}^T x = 0\}$

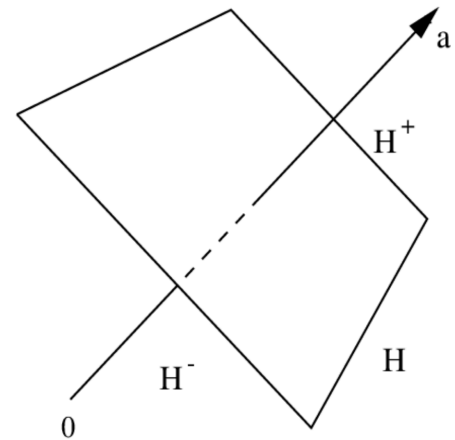


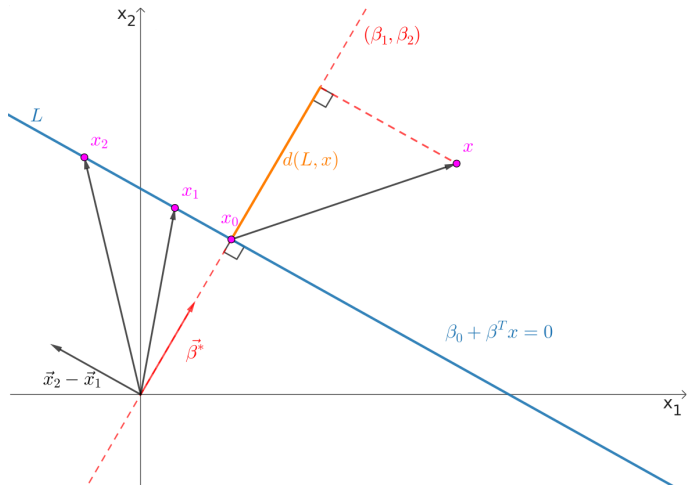
Image source: https://www.researchgate.net/publication/228734473_Linear_conic_programming

- p-dimensional space can be partitioned into:

$$\underbrace{\{x \mid \beta_0 + \beta^T x < 0\}}_{\text{half-plane}} \cup \underbrace{\{x \mid \beta_0 + \beta^T x = 0\}}_{\text{hyper-plane}} \cup \underbrace{\{x \mid \beta_0 + \beta^T x > 0\}}_{\text{half-plane}}$$

Linear Algebra of a Hyperplane

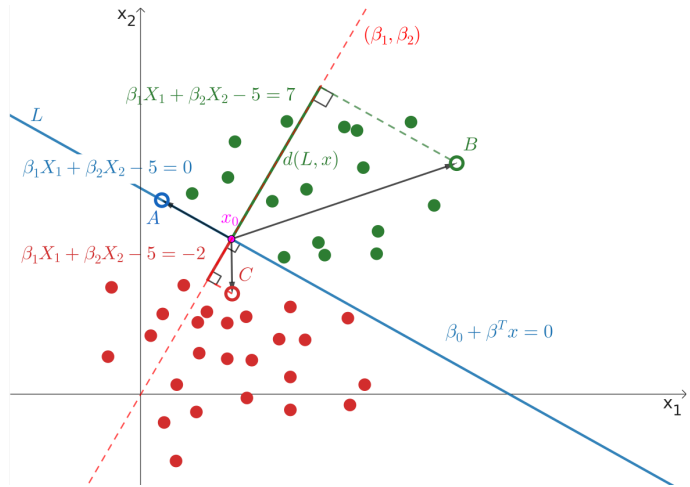
- Affine set L (a hyperplane) is defined as $L := \{x_{2\times 1} \mid \beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0\}$
- Properties of L:
  - $\forall x_1, x_2 \in L$ we have $\beta^T(x_1 - x_2) = 0$
    - i.e. $\vec{\beta}$ is **orthogonal** to L
      - So, $\vec{\beta}^* := \frac{\vec{\beta}}{||\vec{\beta}||}$ is a **normal** vector to L
    - $\forall x_0 \in L$, we have $\beta^T x_0 = -\beta_0$
    - $\forall x \in \mathbb{R}^p$, the **signed distance** is $d(L, x) = \beta^{*T}(x - x_0) \propto \beta^T(x - x_0)$



Based on ESL Fig. 4.15

Linear Algebra of a Hyperplane

- Affine set L (a hyperplane) is defined as $L := \{x_{2\times 1} \mid \beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0\}$
- Properties of L:
  - $\forall x_1, x_2 \in L$ we have $\beta^T(x_1 - x_2) = 0$
    - i.e. $\vec{\beta}$ is **orthogonal** to L
      - So, $\vec{\beta}^* := \frac{\vec{\beta}}{||\vec{\beta}||}$ is a **normal** vector to L
    - $\forall x_0 \in L$, we have $\beta^T x_0 = -\beta_0$
    - $\forall x \in \mathbb{R}^p$, the **signed distance** is $d(L, x) = \beta^{*T}(x - x_0) \propto \beta^T(x - x_0)$



Based on ESL Fig. 4.15

Separating Hyperplanes

- We want to find a **separating hyperplane** (if possible), which perfectly separates the observations of two classes, $y = \pm 1$

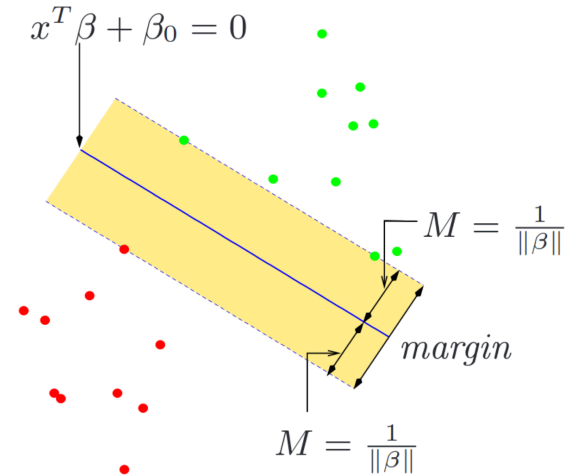
$$\underbrace{\{x \mid \beta_0 + \beta^T x < 0\}}_{\text{y = -1}} \cup \underbrace{\{x \mid \beta_0 + \beta^T x = 0\}}_{\text{hyper-plane}} \cup \underbrace{\{x \mid \beta_0 + \beta^T x > 0\}}_{\text{y = 1}}$$

- **Training**: estimate β_i parameters as $\hat{\beta}_i$
- **Inference**: classify new x^* as $\text{sign}(\beta_0 + \beta^T x^*)$
  - Distance of $x^*_{p \times 1}$ to the hyperplane, $|\beta_0 + \beta^T x^*|$, indicates confidence of prediction

Maximal Margin Classifier (MMC)

Goal: Find the **widest street** between classes

- Among all separating hyperplanes, pick the one **farthest** from the data
- Points closest to boundary = **support vectors**
  - They define the decision boundary
  - All other points are irrelevant!



Source: ESL Fig. 12.1

Optimization:

$$\underset{\beta}{\operatorname{argmax}}\{M \mid y_i(\beta_0 + \beta' x_i) \geq M,\ ||\beta|| = 1\}$$

MMC: Properties and Limitations

Strengths:

- Unique solution (convex optimization)
- Boundary depends **only on support vectors**
  - Move non-support point → boundary unchanged!
- Intuition: widest possible "street"

Limitations:

- Requires **perfect linear separability**
- **Sensitive to outliers**
  - One outlier can drastically shift boundary
- **High variance** — small data changes → big model changes
- Narrow margin → **overfitting**

> **Problem**: Real data is almost never perfectly separable. We need **soft margins → SVC**

Frank Rosenblatt's Perceptron (1958)

- Developed by Rosenblatt 2 years after his PhD'56 at Cornell Aeronautical Laboratory
- Perceptron is a binary classifier with a **hyperplane decision boundary**
- The algorithm minimizes the distance of misclassifications to the hyperplane
- **Goal**: minimize $D(\beta, \beta_0) := -\sum_{i \in \mathcal{M}} y_i(\beta_0 + \beta' x_i)$
  - where $\mathcal{M}$ is the set of misclassified points

Image source: https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon

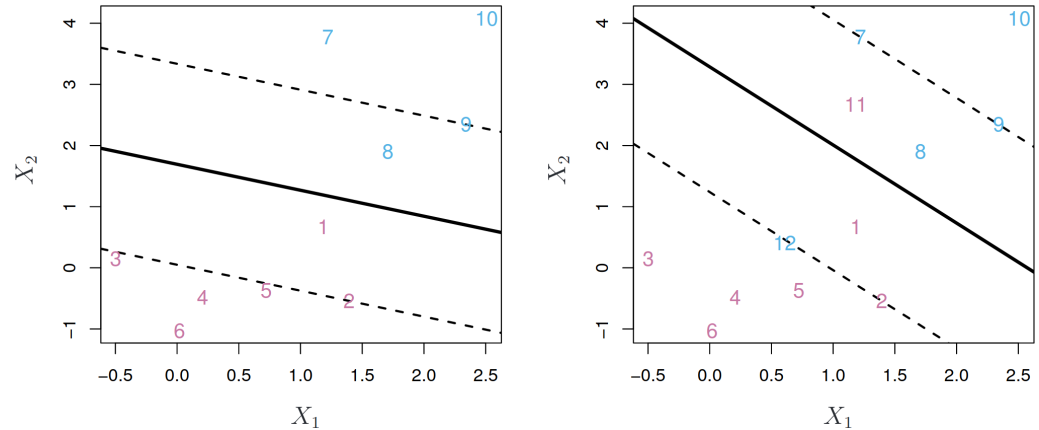
Support Vector Classifier (SVC)

Why SVC? MMC problems:

- Doesn't exist if classes overlap
- Extremely sensitive to outliers

Solution: Allow some violations!

- **Slack variables** ϵ_i measure violations
- **Budget** C limits total violations



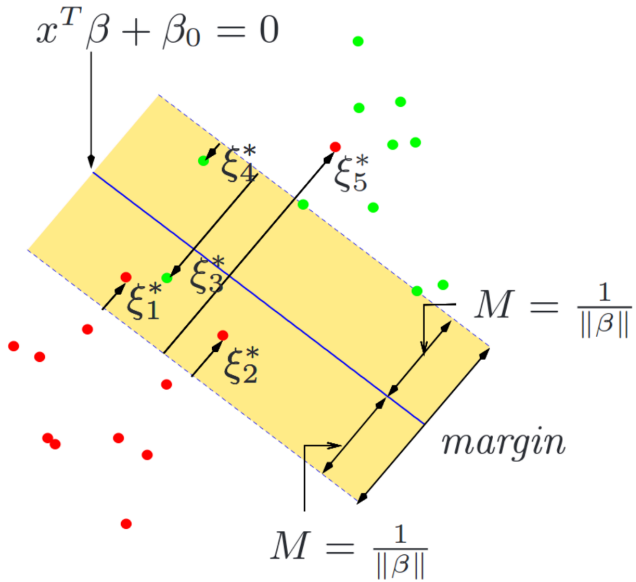
Source: ISLP Fig. 9.6

Slack Variables Explained

What does ϵ_i tell us?

ϵ_i	Point location
$= 0$	Correct side of margin ✓
$\in (0, 1)$	Inside margin (violation)
> 1	Wrong side of hyperplane ✗

- C = "budget" for violations
- At most C points can cross hyperplane
- **Support vectors**: points with $\epsilon_i > 0$



Source: ESL Fig. 12.1

The C Parameter: Bias-Variance Tradeoff

Small C (tight budget, more tolerance)

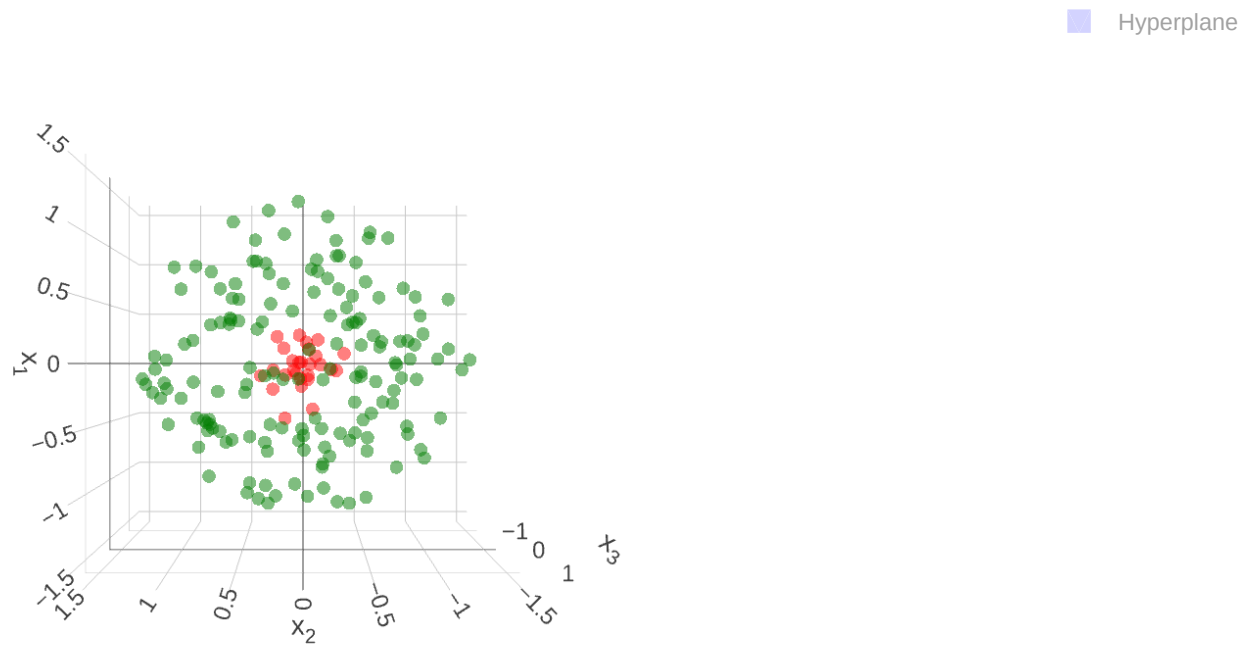
- Wider margin
- More violations allowed
- More support vectors
- **Higher bias, lower variance**
- More robust to outliers
- Similar to **strong regularization**

Large C (generous budget, less tolerance)

- Narrow margin
- Fewer violations allowed
- Fewer support vectors
- **Lower bias, higher variance**
- Sensitive to individual points
- Similar to **weak regularization**

Practical tip: always tune C via cross-validation. In `sklearn`, use `SVC(C=...)` with `GridSearchCV` or `RandomizedSearchCV`

Support Vector Machines: Motivation



Support Vector Machines (SVM)

- We can further extend SVC by adding non-linear terms $X_i^d, X_i X_j, \ldots$
  - However, optimization requires significant computing resources
- Instead, we can use a **kernel trick**
- Consider a **linear kernel** for $u, v \in \mathbb{R}^p$: $K(u, v) = \langle u, v \rangle = \sum_{j=1}^{p} u_j v_j = u \cdot v = u'v$
- It turns out that SVC optimization can be rewritten via $f(x) = \sum_{i=1}^{n} \alpha_i K(x, x_i)$
  - To estimate $\alpha_{1:n}$ and β_0 we need $\frac{n(n-1)}{2}$ **inner products**... , however...
  - $\alpha \neq 0$ only for the support vectors
    - So, estimation simplifies to $f(x) = \sum_{i \in S}^{n} \alpha_i K(x, x_i)$,

The Kernel Trick: Intuition

The problem:

- SVC gives linear boundaries only
- Many real datasets need **non-linear** boundaries

The idea:

- Map data to a **higher-dimensional space** where it becomes linearly separable
- Use a **kernel function** $K(u, v)$ that computes the inner product in the new space **without** explicitly transforming the data

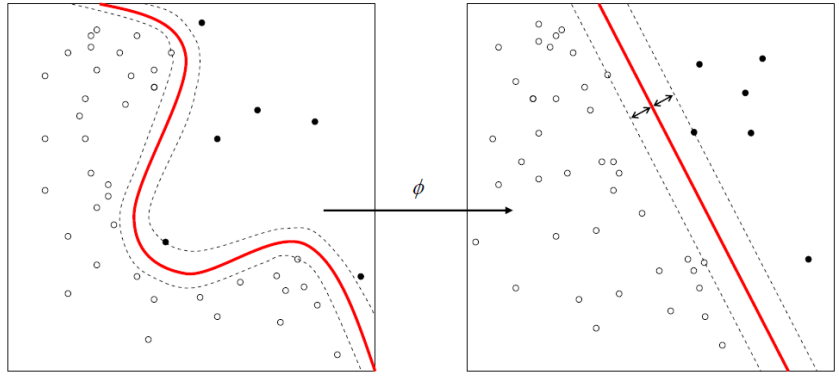


Image source: https://ipython-books.github.io/85-using-support-vector-machines-for-classification-tasks/

Key insight: Kernel computes $K(u, v) = \phi(u)^T \phi(v)$ without ever computing $\phi(u)$ or $\phi(v)$ explicitly. The feature space can even be **infinite-dimensional** (RBF kernel)!

Kernels in Machine Learning

What is a kernel?

A function $K(x, x')$ that measures **similarity** between two points

- Equivalent to inner product in some (often implicit) feature space:

$$K(x, x') = \phi(x)^T \phi(x')$$

- Symmetric: $K(x, x') = K(x', x)$
- Must satisfy Mercer's condition (positive semi-definite)

Why kernels are powerful:

- Compute similarity in **transformed** space
- Without explicitly computing $\phi(x)$
- Feature space can be **infinite-dimensional**!

Common SVM Kernels

- **Polynomial kernel** of degree d:

$$K(u, v) = (1 + \sum_{j=1}^{p} u_j v_j)^d$$

- **Radial (basis) kernel** of degree d:

$$K(u, v) = \exp\Big[-\gamma \sum_{j=1}^{p} (u_j - v_j)^2 \Big] = \exp\Big[-\gamma ||u_j - v_j||_2^2 \Big] \quad \text{for} \quad \gamma > 0$$

- **Neural network kernel**:

$$K(u, v) = \tanh\Big[a\langle u, v\rangle + r \Big]$$

Radial Basis Kernel, $\exp\left[-\gamma||u_j - v_j||_2^2\right]$

- It is essentially an unscaled Gaussian PDF
- Large $u - v$ implies near zero exponent
- So, in $f(x) = \beta_0 + \sum_{i \in S}^{n} \alpha_i K(x, x_i)$
  - $\alpha = 0$ for non-support vectors, or
  - $K(x, x_i) \approx 0$ for distant observations x_i

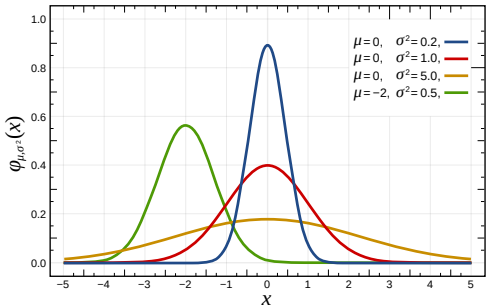


Image source: https://en.wikipedia.org/wiki/Normal_distribution

- Hence, only neighboring support vectors participate in forming a decision boundary

Ex. SVM vs LDA on Heart Disease Data

- Both SVM & LDA compute scores of the form

$$\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \ldots + \hat{\beta}_p X_p$$

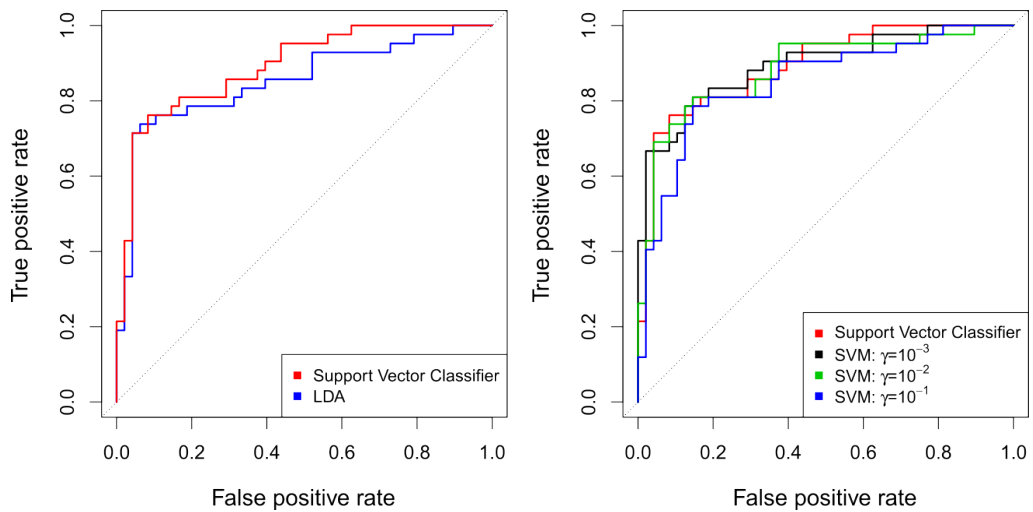


Image source: ISLP Fig. 9.11

SVM is a Binary Classifier

- ... but can be extended to multiclass.
- Consider $K > 2$ classes and a test observation x^*
- **One-versus-one** (OVO) approach
  - Fit $\binom{K}{2}$ pairwise SVMs
  - Assign x^* to the most frequently predicted class
  - Disadvantage: lots of models to fit and inference with
- **One-versus-all** (OVA) approach
  - Fit K SVMs for one of K classes versus all other $K-1$ classes combined
  - Assign x^* to the class with greatest $\beta_{0k} + \beta'_{1k}x_1^* + \ldots + \beta'_{pk}x_p^*$ value
    - Where β_{ik} are fitted for each class $k = 1 : K$
  - Disadvantage: even if we start with **balanced** K classes, we will face **imbalanced** aggregated classes, which will require probability threshold tuning

SVM vs Logistic Regression

- Another form for SVM is $\underset{\beta_{0:p}}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} \underbrace{\max[0, 1 - y_i f(x_i)]}_{\text{Hinge loss}} + \underbrace{\lambda \sum_{j=1}^{p} \beta_j^2}_{\text{Ridge penalty}} \right\},$

where $\lambda \geq 0$ and correlates with the parameter C

- Hinge loss is similar to logistic regression loss
- Changing the loss function yields a regularized logistic regression

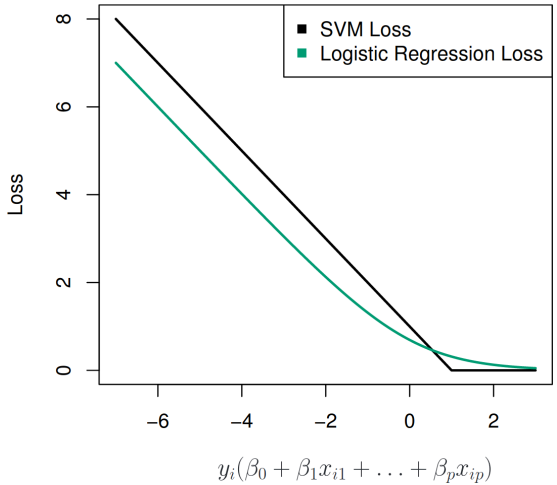


Image source: ISLR Fig. 9.12

Practical Guide: Choosing Your SVM

Scenario	Recommended Approach
n large, p small	RBF kernel, tune C and γ
$p \gg n$ (e.g., text)	Linear kernel or Logistic Regression
Want probabilities	Logistic Regression or `SVC(probability=True)`
Non-linear boundary	RBF kernel (default choice)
Very large n (>100k)	Consider `SGDClassifier(loss='hinge')`

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Always scale features before SVM!
```

SVM Playgrounds

- http://macheads101.com/demos/svm-playground
- https://ml-playground.com
- https://mlplaygrounds.com/machine/learning/svm.html
- https://greitemann.dev/svm-demo

- Examine also Perceptron playground:
  - https://perceptrondemo.com

When to Use SVM?

SVM shines when:

- Small to medium-sized structured dataset ($n < 100\text{k}$)
- Number of features is large (even $p > n$)
  - e.g., text classification, genomics
- You need a non-linear decision boundary
- Clear margin of separation exists
- You want a well-regularized model

Consider alternatives when:

- Dataset is very large ($n > 100\text{k}$) → use logistic regression or neural networks
- You need probability estimates → logistic regression
- Interpretability is important → decision trees, logistic regression
- Many categorical features → tree-based methods
- You want to avoid extensive hyperparameter tuning

Key Takeaways

Concepts:

1. **Hyperplane** divides space into two half-spaces
2. **MMC** = widest separating margin (hard margin)
3. **SVC** = soft margin with slack variables ϵ_i
4. **SVM** = kernel trick for non-linear boundaries

Practical tips:

5. **Always standardize** features before SVM
6. Start with **RBF kernel** (default choice)
7. Use **linear kernel** for sparse, high-dim data
8. **Tune C and γ** via cross-validation!

> **Remember**: SVM = "widest street" + kernels. Simple, elegant, still useful!

END